

Employee API Guide Study

Project: EmployeeApi

Generated: September 8, 2026

Purpose: full step-by-step guide with copy-paste code for role login, employee password hashing, JWT authorization, Dapper stored procedures, migrations, SMTP email, and testing.

Security note: this document intentionally uses placeholder SMTP credentials. Do not paste a real Gmail app password into documentation or commit it to a public repository. If a real app password was shared, rotate it in Google account settings.

1. Big Picture

The backend is an ASP.NET Core Web API. It receives JSON from frontend/Postman, calls services, talks to SQL Server, and returns JSON.

Frontend/Postman

- > Controller
- > Service Interface
- > Service Implementation
- > EF Core or Dapper Stored Procedure
- > SQL Server
- > JSON response

For login:

- ```
POST /api/auth/login
-> AuthApiController
-> IEmployeeService.GetByEmail()
-> EmployeeService
-> Dapper
-> dbo.GetEmployeeByEmail
-> PasswordHasher verifies password
-> JWT token returned
```

## 2. Folder Structure

```
EmployeeApi/
 Api/v1/
 AuthApiController.cs
 EmployeeApiController.cs
 AssignTaskApiController.cs
 DepartmentApiController.cs
 ClientApiController.cs

 Configuration/
 MailSettings.cs

 Data/
 ApplicationDbContext.cs
 EmployeeApiDatabaseInitializer.cs

 Model/
 Employee/Employee.cs
 Department/Department.cs
 Client/Client.cs
 AssignTask/AssignedTask.cs

 Service/
 Employee/IEmployeeService.cs
 Employee/EmployeeService.cs
 Notifications/IEmployeeEmailSender.cs
 Notifications/SmtEmployeeEmailSender.cs
 Department/
 Client/
 AssignTask/

 ViewModel/
 Auth/LoginRequestViewModel.cs
 Auth/LoginResponseViewModel.cs
 Employee/CreateEmployeeViewModel.cs
 Employee/UpdateEmployeeViewModel.cs

 Migrations/
 EF Core migration files

 Documents/
 guide_study.html
 guide_study.pdf

 appsettings.json
 Program.cs
 EmployeeApiServiceRegistrar.cs
 EmployeeApi.csproj
```

## 3. Step 1: Install Dapper

---

Dapper is used to call SQL stored procedures directly from C#.

```
dotnet add EmployeeApi package Dapper
```

After install, `EmployeeApi.csproj` should contain:

```
<PackageReference Include="Dapper" Version="2.1.79" />
```

## 4. Step 2: Employee Model

---

File: `EmployeeApi/Model/Employee/Employee.cs`

This model maps to the `Employees` table. Use `PasswordHash`, not plain `Password`, in the database model.

```
using System.Text.Json.Serialization;

namespace EmployeeApi.Model.Employee
{
 public class Employee
 {
 public int Id { get; set; }

 public string Name { get; set; } = string.Empty;

 public string Email { get; set; } = string.Empty;

 public string PhoneNumber { get; set; } = string.Empty;

 public decimal Salary { get; set; }

 public int DepartmentId { get; set; }

 public int? ClientId { get; set; }

 [JsonIgnore]
 public string PasswordHash { get; set; } = string.Empty;

 public string Role { get; set; } = "Employee";

 public EmployeeApi.Model.Client.Client? Client { get; set; }

 public EmployeeApi.Model.Department.Department? Department { get; set; }
 }
}
```

**Why JsonIgnore?** It prevents PasswordHash from being returned in API JSON responses.

## 5. Step 3: Create Employee ViewModel

File: EmployeeApi/ViewModel/Employee/CreateEmployeeViewModel.cs

The frontend sends plain password here. The controller hashes it before saving.

```
using System.ComponentModel.DataAnnotations;

namespace EmployeeApi.ViewModel.Employee;

public class CreateEmployeeViewModel
{
 [Required]
 [StringLength(150)]
 public string Name { get; set; } = string.Empty;

 [Required]
 [EmailAddress]
 [StringLength(255)]
 public string Email { get; set; } = string.Empty;

 [StringLength(10)]
 [Phone]
 public string? PhoneNumber { get; set; }

 [Range(0, double.MaxValue)]
 public decimal Salary { get; set; }

 public int? ClientId { get; set; }

 public int DepartmentId { get; set; }

 [Required]
 [StringLength(100, MinimumLength = 6)]
 public string Password { get; set; } = string.Empty;

 [Required]
 [RegularExpression("^(Manager|Employee)$", ErrorMessage = "Role must be Manager or Employee.")
 public string Role { get; set; } = "Employee";
}
```

## 6. Step 4: Update Employee ViewModel

File: `EmployeeApi/ViewModel/Employee/UpdateEmployeeViewModel.cs`

Do not inherit from create model if password should be optional during update.

```
using System.ComponentModel.DataAnnotations;

namespace EmployeeApi.ViewModel.Employee;

public class UpdateEmployeeViewModel
{
 [Required]
 [StringLength(150)]
 public string Name { get; set; } = string.Empty;

 [Required]
 [EmailAddress]
 [StringLength(255)]
 public string Email { get; set; } = string.Empty;

 [StringLength(10)]
 [Phone]
 public string? PhoneNumber { get; set; }

 [StringLength(100, MinimumLength = 6)]
 public string? Password { get; set; }

 [RegularExpression("^(Manager|Employee)$", ErrorMessage = "Role must be Manager or Employee.")
 public string? Role { get; set; }

 [Range(0, double.MaxValue)]
 public decimal Salary { get; set; }

 public int? ClientId { get; set; }

 public int DepartmentId { get; set; }
}
```

## 7. Step 5: Migration For PasswordHash And Role

Migration is needed only because the database table needs new columns.

```
dotnet ef migrations add addrolebaseauth --project EmployeeApi
dotnet ef database update --project EmployeeApi
```

Expected migration idea:

```
migrationBuilder.AddColumn<string>(
 name: "PasswordHash",
 table: "Employees",
 type: "nvarchar(max)",
 nullable: false,
 defaultValue: "");

migrationBuilder.AddColumn<string>(
 name: "Role",
 table: "Employees",
 type: "nvarchar(max)",
 nullable: false,
 defaultValue: "Employee");
```

Check SQL Server:

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Employees';
```

## 8. Step 6: Stored Procedures In SQL Server

---

Create these in SSMS inside the same database used by the API: `EmployeeApiDb` .

### 8.1 Get Employee By Email

```
USE EmployeeApiDb;
GO

CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeByEmail]
 @Email NVARCHAR(255)
AS
BEGIN
 SET NOCOUNT ON;

 SELECT
 Id,
 Name,
 Email,
 PhoneNumber,
 PasswordHash,
 Role,
 Salary,
 DepartmentId,
 ClientId
 FROM Employees
 WHERE Email = @Email;
END
GO
```

## 8.2 Get All Employees

```
USE EmployeeApiDb;
GO

CREATE OR ALTER PROCEDURE [dbo].[GetAllEmployees]
AS
BEGIN
 SET NOCOUNT ON;

 SELECT
 Id,
 Name,
 Email,
 PhoneNumber,
 PasswordHash,
 Role,
 Salary,
 DepartmentId,
 ClientId
 FROM Employees
 ORDER BY Id DESC;
END
GO
```

## 8.3 Get Employee By Id

```
USE EmployeeApiDb;
GO

CREATE OR ALTER PROCEDURE [dbo].[GetEmployeeById]
 @Id INT
AS
BEGIN
 SET NOCOUNT ON;

 SELECT
 Id,
 Name,
 Email,
 PhoneNumber,
 PasswordHash,
 Role,
 Salary,
 DepartmentId,
 ClientId
 FROM Employees
 WHERE Id = @Id;
END
GO
```

## 8.4 Delete Employee

```
USE EmployeeApiDb;
GO

CREATE OR ALTER PROCEDURE [dbo].[DeleteEmployee]
 @Id INT
AS
BEGIN
 SET NOCOUNT ON;

 DELETE FROM Employees
 WHERE Id = @Id;

 SELECT @@ROWCOUNT;
END
GO
```

## 8.5 Test Stored Procedures

```
EXEC [dbo].[GetEmployeeByEmail] @Email = 'employee@example.com';
EXEC [dbo].[GetAllEmployees];
EXEC [dbo].[GetEmployeeById] @Id = 1;
EXEC [dbo].[DeleteEmployee] @Id = 1;
```

## 9. Step 7: Employee Service Interface

---

File: EmployeeApi/Service/Employee/IEmployeeService.cs

```
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Employee
{
 public interface IEmployeeService
 {
 Task<List<EmployeeEntity>> GetAll();

 Task<EmployeeEntity?> GetById(int id);

 Task<EmployeeEntity> Create(EmployeeEntity employee);

 Task<EmployeeEntity> Update(EmployeeEntity employee);

 Task<bool> Delete(int id);

 Task<EmployeeEntity?> GetByEmail(string email);
 }
}
```

## 10. Step 8: Employee Service With Dapper

---

File: EmployeeApi/Service/Employee/EmployeeService.cs

This version uses Dapper for list, get by id, get by email, and delete. It keeps EF Core for create and update.

```
using Dapper;
using EmployeeApi.Data;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;
using System.Data;
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Employee
{
 public class EmployeeService : IEmployeeService
 {
 private readonly ApplicationDbContext _context;
 private readonly string _connString;

 public EmployeeService(ApplicationDbContext context, IConfiguration configuration)
 {
 _context = context;
 _connString = configuration.GetConnectionString("DefaultConnection")
 ?? throw new InvalidOperationException("Connection string not found.");
 }

 public async Task<EmployeeEntity?> GetByEmail(string email)
 {
 using var connection = new SqlConnection(_connString);

 return await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
 "[dbo].[GetEmployeeByEmail]",
 new { Email = email },
 CommandType.StoredProcedure);
 }

 public async Task<List<EmployeeEntity>> GetAll()
 {
 using var connection = new SqlConnection(_connString);

 var employees = await connection.QueryAsync<EmployeeEntity>(
 "[dbo].[GetAllEmployees]",
 CommandType.StoredProcedure);

 return employees.ToList();
 }

 public async Task<EmployeeEntity?> GetById(int id)
 {
 using var connection = new SqlConnection(_connString);

 return await connection.QueryFirstOrDefaultAsync<EmployeeEntity>(
 "[dbo].[GetEmployeeById]",
 new { Id = id },
 CommandType.StoredProcedure);
 }

 public async Task<EmployeeEntity> Create(EmployeeEntity employee)
```

```

 {
 _context.Employees.Add(employee);
 await _context.SaveChangesAsync();
 return employee;
 }

 public async Task<EmployeeEntity> Update(EmployeeEntity employee)
 {
 var existingEmployee = await _context.Employees.FindAsync(employee.Id)
 ?? throw new KeyNotFoundException($"Employee {employee.Id} was not found.");

 _context.Entry(existingEmployee).CurrentValues.SetValues(employee);
 await _context.SaveChangesAsync();
 return existingEmployee;
 }

 public async Task<bool> Delete(int id)
 {
 using var connection = new SqlConnection(_connString);

 var affectedRows = await connection.ExecuteScalarAsync<int>(
 "[dbo].[DeleteEmployee]",
 new { Id = id },
 CommandType.StoredProcedure);

 return affectedRows > 0;
 }
}

```

## 11. Step 9: Login Request And Response ViewModels

File: EmployeeApi/ViewModel/Auth/LoginRequestViewModel.cs

```

namespace EmployeeApi.ViewModel.Auth;

public class LoginRequestViewModel
{
 public string Username { get; set; } = string.Empty;

 public string Password { get; set; } = string.Empty;
}

```

File: EmployeeApi/ViewModel/Auth/LoginResponseViewModel.cs

```
namespace EmployeeApi.ViewModel.Auth;

public class LoginResponseViewModel
{
 public string Token { get; set; } = string.Empty;

 public DateTime ExpiresAt { get; set; }
}
```

## 12. Step 10: Auth Controller

---

File: EmployeeApi/Api/v1/AuthApiController.cs

This controller supports temporary manager login and real employee login.

```
using EmployeeApi.Model.Employee;
using EmployeeApi.Service.Employee;
using EmployeeApi.ViewModel.Auth;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace EmployeeApi.Api.v1;

[ApiController]
[Route("api/auth")]
public class AuthApiController : ControllerBase
{
 private readonly IConfiguration _configuration;
 private readonly IEmployeeService _employeeService;

 public AuthApiController(IConfiguration configuration, IEmployeeService employeeService)
 {
 _configuration = configuration;
 _employeeService = employeeService;
 }

 [AllowAnonymous]
 [HttpPost("login")]
 public async Task<ActionResult> Login([FromBody] LoginRequestViewModel request)
 {
 var configuredUsername = _configuration["ApiUser:Username"];
 var configuredPassword = _configuration["ApiUser:Password"];

 if (request.Username == configuredUsername && request.Password == configuredPassword)
 {
 var adminToken = CreateToken(request.Username, "Manager");
 return Ok(adminToken);
 }

 var employee = await _employeeService.GetByEmail(request.Username);

 if (employee is null)
 {
 return Unauthorized("Invalid username or password.");
 }

 var hasher = new PasswordHasher<Employee>();
 var passwordResult = hasher.VerifyHashedPassword(
 employee,
 employee.PasswordHash,
 request.Password);

 if (passwordResult == PasswordVerificationResult.Failed)
```

```
{
 return Unauthorized("Invalid username or password.");
}

var token = CreateToken(employee.Email, employee.Role, employee.Id);
return Ok(token);
}

private LoginResponseViewModel CreateToken(string username, string role, int? employeeId = null)
{
 var secret = _configuration["Jwt:Secret"]
 ?? throw new InvalidOperationException("Jwt:Secret is not configured.");
 var issuer = _configuration["Jwt:Issuer"];
 var audience = _configuration["Jwt:Audience"];
 var expirationMinutes = _configuration.GetValue("Jwt:ExpirationMinutes", 30);
 var expiresAt = DateTime.UtcNow.AddMinutes(expirationMinutes);

 var claims = new List<Claim>
 {
 new Claim(JwtRegisteredClaimNames.Sub, username),
 new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
 new Claim(ClaimTypes.Name, username),
 new Claim(ClaimTypes.Role, role)
 };

 if (employeeId.HasValue)
 {
 claims.Add(new Claim(ClaimTypes.NameIdentifier, employeeId.Value.ToString()));
 }

 var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret));
 var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
 var jwt = new JwtSecurityToken(
 issuer: issuer,
 audience: audience,
 claims: claims,
 expires: expiresAt,
 signingCredentials: credentials);

 return new LoginResponseViewModel
 {
 Token = new JwtSecurityTokenHandler().WriteToken(jwt),
 ExpiresAt = expiresAt
 };
}
```

## 13. Step 11: Employee Controller With Password Hashing And Email

---

File: `EmployeeApi/Api/v1/EmployeeApiController.cs`

```
using EmployeeApi.Service.Employee;
using EmployeeApi.Service.Notifications;
using EmployeeApi.ViewModel.Employee;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Api.v1
{
 [ApiController]
 [Route("api/employee")]
 public class EmployeeApiController : ControllerBase
 {
 private readonly IEmployeeService _service;
 private readonly IEmployeeEmailSender _employeeEmailSender;

 public EmployeeApiController(
 IEmployeeService service,
 IEmployeeEmailSender employeeEmailSender)
 {
 _service = service;
 _employeeEmailSender = employeeEmailSender;
 }

 [HttpGet]
 [Authorize(Roles = "Manager")]
 public async Task<IActionResult> Get()
 {
 return Ok(await _service.GetAll());
 }

 [HttpGet("{id:int}")]
 [Authorize(Roles = "Manager,Employee")]
 public async Task<IActionResult> GetById(int id)
 {
 var employee = await _service.GetById(id);
 return employee is null ? NotFound() : Ok(employee);
 }

 [HttpPost("new")]
 [Authorize(Roles = "Manager")]
 public async Task<IActionResult> Create([FromBody] CreateEmployeeViewModel model)
 {
 var employee = new EmployeeEntity
 {
 Name = model.Name,
 Email = model.Email,
 PhoneNumber = model.PhoneNumber ?? string.Empty,
 Salary = model.Salary,
 ClientId = model.ClientId,
 DepartmentId = model.DepartmentId,
 Role = model.Role
 };
 await _service.Create(employee);
 return CreatedAtRoute("api/employee/{id}", new { id = employee.Id }, employee);
 }
 }
}
```

```
};

var hasher = new PasswordHasher<EmployeeEntity>();
employee.PasswordHash = hasher.HashPassword(employee, model.Password);

var createdEmployee = await _service.Create(employee);

await _employeeEmailSender.SendAccountCreatedAsync(
 createdEmployee,
 model.Password);

return CreatedAtAction(
 nameof(GetById),
 new { id = createdEmployee.Id },
 createdEmployee);
}

[HttpPut("{id:int}")]
[Authorize(Roles = "Manager")]
public async Task<IActionResult> Update(int id, [FromBody] UpdateEmployeeViewModel model)
{
 var existingEmployee = await _service.GetById(id);
 if (existingEmployee is null)
 {
 return NotFound();
 }

 var employee = new EmployeeEntity
 {
 Id = id,
 Name = model.Name,
 Email = model.Email,
 PhoneNumber = model.PhoneNumber ?? string.Empty,
 PasswordHash = existingEmployee.PasswordHash,
 Role = string.IsNullOrEmpty(model.Role) ? existingEmployee.Role : model.Role,
 Salary = model.Salary,
 ClientId = model.ClientId,
 DepartmentId = model.DepartmentId
 };

 if (!string.IsNullOrEmpty(model.Password))
 {
 var hasher = new PasswordHasher<EmployeeEntity>();
 employee.PasswordHash = hasher.HashPassword(employee, model.Password);
 }

 return Ok(await _service.Update(employee));
}

[HttpDelete("{id:int}")]
[Authorize(Roles = "Manager")]
public async Task<IActionResult> Delete(int id)
{
 return await _service.Delete(id) ? NoContent() : NotFound();
}
```

```
}
}
}
```

## 14. Step 12: Protect Other Controllers

---

If the whole controller is Manager-only, add this at the top:

```
using Microsoft.AspNetCore.Authorization;
```

Then place this above the class:

```
[Authorize(Roles = "Manager")]
```

### AssignTaskApiController Example

```
[ApiController]
[Route("api/assign-task")]
[Authorize(Roles = "Manager")]
public class AssignTaskApiController : ControllerBase
{
 // All actions here are Manager-only.
}
```

### DepartmentApiController Example

```
[ApiController]
[Route("api/department")]
[Authorize(Roles = "Manager")]
public class DepartmentApiController : ControllerBase
{
 // All actions here are Manager-only.
}
```

### ClientApiController Example

```
[ApiController]
[Route("api/client")]
[Authorize(Roles = "Manager")]
public class ClientApiController : ControllerBase
{
 // All actions here are Manager-only.
}
```

## 15. Step 13: Mail Settings

---

File: EmployeeApi/Configuration/MailSettings.cs

```
namespace EmployeeApi.Configuration;

public class MailSettings
{
 public bool Enabled { get; set; }

 public string Host { get; set; } = string.Empty;

 public int Port { get; set; }

 public bool UseSsl { get; set; }

 public string UserName { get; set; } = string.Empty;

 public string Password { get; set; } = string.Empty;

 public string FromEmail { get; set; } = string.Empty;

 public string FromName { get; set; } = string.Empty;
}
```

## 16. Step 14: Email Sender Interface

---

File: EmployeeApi/Service/Notifications/IEmployeeEmailSender.cs

```
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Notifications;

public interface IEmployeeEmailSender
{
 Task SendAccountCreatedAsync(EmployeeEntity employee, string plainPassword);
}
```

## 17. Step 15: SMTP Email Sender

---

File: EmployeeApi/Service/Notifications/SmtpEmployeeEmailSender.cs

```
using EmployeeApi.Configuration;
using Microsoft.Extensions.Options;
using System.Net;
using System.Net.Mail;
using EmployeeEntity = EmployeeApi.Model.Employee.Employee;

namespace EmployeeApi.Service.Notifications;

public class SmtplibEmployeeEmailSender : IEmployeeEmailSender
{
 private readonly MailSettings _settings;

 public SmtplibEmployeeEmailSender(IOptions<MailSettings> options)
 {
 _settings = options.Value;
 }

 public async Task SendAccountCreatedAsync(EmployeeEntity employee, string plainPassword)
 {
 if (!_settings.Enabled)
 {
 return;
 }

 using var message = new MailMessage
 {
 From = new MailAddress(_settings.FromEmail, _settings.FromName),
 Subject = "Your employee account has been created",
 Body = $""
Hello {employee.Name},

Your employee account has been created.

Login Email: {employee.Email}
Password: {plainPassword}
Role: {employee.Role}

Please change your password after login.
""",
 IsBodyHtml = false
 };

 message.To.Add(employee.Email);

 using var smtpClient = new SmtpClient(_settings.Host, _settings.Port)
 {
 EnableSsl = _settings.UseSsl,
 Credentials = new NetworkCredential(_settings.UserName, _settings.Password)
 };

 await smtpClient.SendMailAsync(message);
 }
}
```

```
}
}
```

## 18. Step 16: Register Services In DI

File: EmployeeApi/EmployeeApiServiceRegistrar.cs

```
using EmployeeApi.Service.AssignTask;
using EmployeeApi.Service.Client;
using EmployeeApi.Service.Department;
using EmployeeApi.Service.Employee;
using EmployeeApi.Service.Notifications;

namespace EmployeeApi;

public static class EmployeeApiServiceRegistrar
{
 public static IServiceCollection AddEmployeeApiServices(this IServiceCollection services)
 {
 services.AddScoped<IEmployeeService, EmployeeService>();
 services.AddScoped<IAssignTaskService, AssignTaskService>();
 services.AddScoped<IDepartmentService, DepartmentService>();
 services.AddScoped<IClientService, ClientService>();
 services.AddScoped<IEmployeeEmailSender, SmtplibEmployeeEmailSender>();

 return services;
 }
}
```

## 19. Step 17: Program.cs Setup

Important snippets for Program.cs .

```
using EmployeeApi.Configuration;
```

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
 options.UseSqlServer(
 builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.Configure<MailSettings>(
 builder.Configuration.GetSection("MailSettings"));

builder.Services.AddEmployeeApiServices();
```

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
 .AddJwtBearer(options =>
 {
 options.TokenValidationParameters = new TokenValidationParameters
 {
 ValidateIssuer = true,
 ValidateAudience = true,
 ValidateLifetime = true,
 ValidateIssuerSigningKey = true,
 ValidIssuer = builder.Configuration["Jwt:Issuer"],
 ValidAudience = builder.Configuration["Jwt:Audience"],
 IssuerSigningKey = new SymmetricSecurityKey(
 Encoding.UTF8.GetBytes(jwtSecret)),
 ClockSkew = TimeSpan.Zero
 };
 });

builder.Services.AddAuthorization();
```

```
app.UseCors("ReactPolicy");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

## 20. Step 18: appsettings.json

---

Use this structure. Replace SMTP values with your provider values.

```
{
 "ConnectionStrings": {
 "DefaultConnection": "Server=localhost;Database=EmployeeApiDb;Trusted_Connection=True;Multiple
 },
 "Logging": {
 "LogLevel": {
 "Default": "Information",
 "Microsoft.AspNetCore": "Warning"
 }
 },
 "Jwt": {
 "Issuer": "EmployeeApi",
 "Audience": "EmployeeApiClient",
 "Secret": "EmployeeApi-Development-Secret-Key-Change-Before-Production-2026",
 "ExpirationMinutes": 1740
 },
 "ApiUser": {
 "Username": "admin",
 "Password": "admin123"
 },
 "MailSettings": {
 "Enabled": true,
 "Host": "smtp.gmail.com",
 "Port": 587,
 "UseSsl": true,
 "UserName": "your-email@gmail.com",
 "Password": "your-gmail-app-password",
 "FromEmail": "your-email@gmail.com",
 "FromName": "Employee API"
 },
 "AllowedHosts": "*"
}
```

## 21. Step 19: Frontend Requirements

Login endpoint:

```
POST /api/auth/login
```

Request body:

```
{
 "username": "employee@example.com",
 "password": "EmployeePassword"
}
```

For employee login, `username` means email.

Protected requests must include:

```
Authorization: Bearer <token>
```

Frontend should decode JWT role and show/hide UI based on:

```
Manager
Employee
```

## 22. Step 20: Testing With Postman

### 22.1 Login As Temporary Manager

```
POST http://localhost:5256/api/auth/login
Content-Type: application/json
```

```
{
 "username": "admin",
 "password": "admin123"
}
```

### 22.2 Create Employee

```
POST http://localhost:5256/api/employee/new
Authorization: Bearer <manager-token>
Content-Type: application/json
```

```
{
 "name": "Test Employee",
 "email": "employee@example.com",
 "phoneNumber": "9800000000",
 "password": "Test1234",
 "role": "Employee",
 "salary": 10000,
 "departmentId": 1,
 "clientId": null
}
```

### 22.3 Login As Employee

```
POST http://localhost:5256/api/auth/login
Content-Type: application/json

{
 "username": "employee@example.com",
 "password": "Test1234"
}
```

## 22.4 Test Manager-Only API With Employee Token

```
GET http://localhost:5256/api/assign-task
Authorization: Bearer <employee-token>
```

Expected result if endpoint is Manager-only:

403 Forbidden

## 23. SQL Debug Queries

---

### Check Columns

```
SELECT COLUMN_NAME, DATA_TYPE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Employees';
```

### Check Employee Login Data

```
SELECT Id, Name, Email, Role, PasswordHash
FROM Employees
WHERE Email = 'employee@example.com';
```

### Check Stored Procedure Exists

```
SELECT
 DB_NAME() AS DatabaseName,
 SCHEMA_NAME(schema_id) AS SchemaName,
 name AS ProcedureName
FROM sys.procedures
WHERE name IN (
 'GetEmployeeByEmail',
 'GetAllEmployees',
 'GetEmployeeById',
 'DeleteEmployee'
);
```

## 24. HTTP Status Codes

Status	Meaning	Example
200 OK	Request successful.	Login success, get list success.
201 Created	New record created.	Employee created.
204 No Content	Delete success with no response body.	Employee deleted.
401 Unauthorized	No valid authentication.	Wrong password, missing token.
403 Forbidden	Logged in but role not allowed.	Employee token on Manager API.
404 Not Found	Record not found.	Employee id does not exist.

## 25. Common Mistakes And Fixes

Mistake	Why It Breaks	Fix
Saving plain password in DB	Login expects hashed password.	Use <code>PasswordHasher.HashPassword</code> .
Using <code>Password</code> column in model	Confuses plain password and hash.	Use <code>PasswordHash</code> .
Stored procedure exists in wrong DB	API cannot find it.	Create SP in <code>EmployeeApiDb</code> .
Role spelling mismatch	Authorize checks exact string.	Use exactly <code>Manager</code> or <code>Employee</code> .
Old JWT token	Token may not contain role.	Login again and use fresh token.
SMTP normal Gmail password	Gmail blocks normal password.	Use Gmail app password.
Backend not restarted	Old code/config still running.	Restart API after changes.

## 26. What Needs Migration And What Does Not

Change	Migration Needed?
Add PasswordHash column	Yes
Add Role column	Yes
Add [Authorize]	No
Change login logic	No
Add Dapper package	No database migration
Create stored procedure manually in SSMS	No EF migration
Add SMTP email sending	No
Add a new database table	Yes

## 27. Final End-To-End Flow

1. Manager logs in with admin/admin123.
2. Backend returns JWT with role Manager.
3. Manager calls POST /api/employee/new.
4. Controller receives CreateEmployeeViewModel.
5. Controller hashes model.Password into Employee.PasswordHash.
6. EmployeeService saves employee to SQL Server.
7. SmtplibEmployeeEmailSender sends credentials to employee email.
8. Employee logs in with email/password.
9. AuthApiController calls EmployeeService.GetByEmail.
10. EmployeeService calls dbo.GetEmployeeByEmail using Dapper.
11. PasswordHasher verifies request password against PasswordHash.
12. Backend returns JWT with role from employee table.
13. Protected controllers allow or block requests based on role.

## 28. Study Checklist

1. Read Employee.cs and understand model vs database.
2. Read CreateEmployeeViewModel.cs and understand request body.
3. Trace EmployeeApiController.Create from request to save to email.
4. Trace AuthApiController.Login from request to token.

5. Run each stored procedure manually in SSMS.
6. Login as manager in Postman.
7. Create employee in Postman.
8. Login as employee in Postman.
9. Test 401 by using wrong password.
10. Test 403 by using employee token on Manager-only endpoint.